

Semester: CE-5

Comsats University Islamabad Attock Campus



Submitted By: Sumaira Safeer
Registration number: FA22-BCE-019
Submitted To: Sobia Irum
Course Name: Operating Systems
Program: BS Computer Engineering
Topic: Project Report

Department of Electrical and Computer Engineering

Implementation and Comparative Analysis of CPU Scheduling Algorithms in C++

Overview:

The provided C++ program demonstrates the implementation of various CPU scheduling algorithms using a Process class. Each process is defined by properties like arrival time, burst time, priority, and runtime metrics. The algorithms implemented are:

1. **First Come First Serve (FCFS).**
2. **Shortest Job First (SJF) - Non-preemptive.**
3. **Shortest Remaining Time First (SRTF) – Preemptive.**
4. **Round Robin (RR).**
5. **Priority Scheduling (Preemptive).**

1. Introduction:

CPU scheduling is essential for optimizing system performance by determining the order in which processes execute on the CPU. This report explores the implementation of five key scheduling algorithms: First Come First Serve (FCFS), Shortest Job First (SJF), Shortest Remaining Time First (SRTF), Round Robin (RR), and Priority Scheduling. Each algorithm is analyzed for its impact on process metrics such as waiting time and turnaround time, demonstrating their suitability for different workloads.

2. Process Class:

The Process class encapsulates the data and behaviors of a single process:

- **Attributes:**
 - **id:** Unique identifier.
 - **arrivalTime:** Time at which the process arrives.
 - **burstTime:** Total CPU time required.
 - **remainingTime:** Remaining CPU time to execute.
 - **priority:** Priority level (lower value = higher priority).
 - **waitTime:** Time spent waiting in the queue.
 - **turnaroundTime:** Total time in the system (completion time - arrival time).
 - **hasStarted:** Tracks if a process has started execution.

- **Methods:**

- Getters for accessing attributes.
- Update methods for decrementing time, setting wait/turnaround times, and marking start.

3. Scheduling Algorithms:

a) First Come First Serve (FCFS):

- **Description:** Processes are scheduled in the order of their arrival.
- **Implementation Highlights:**
 - Iterates through processes based on arrival time.
 - Executes processes one at a time until completion.
- **Output:** Includes the start and completion times, wait time, and turnaround time for each process.

b) Shortest Job First (SJF):

- **Description:** Executes the process with the shortest burst time next.
- **Implementation Highlights:**
 - At each time unit, selects the process with the smallest burst time among the available processes.
 - Non-preemptive; once a process starts, it runs to completion.
- **Output:** Provides detailed scheduling with minimal average wait and turnaround times.

c) Shortest Remaining Time First (SRTF):

- **Description:** Preemptive variant of SJF where the process with the least remaining time is executed first.
- **Implementation Highlights:**
 - Continuously checks for processes with a shorter remaining time and preempts the current process if needed.
- **Output:** Includes process switches for efficient CPU utilization.

d) Round Robin (RR):

- **Description:** Processes are executed in cyclic order for a fixed quantum.
- **Implementation Highlights:**
 - Utilizes a queue to manage process execution.

- Each process is allocated a time quantum (2 units in this case).
- Processes re-enter the queue if not completed within the quantum.
- **Output:** Demonstrates time slicing and fairness across processes.

e) Priority Scheduling:

- **Description:** Executes processes based on priority (lower value = higher priority).
- **Implementation Highlights:**
 - Continuously selects the highest-priority process available.
 - Preemptive; processes can be interrupted by higher-priority arrivals.
- **Output:** Displays execution and context switches due to priority adjustments.

4. Code Implementation:

```
#include <iostream>
#include <vector>
#include <queue>
#include <algorithm>
using namespace std;
// Process class to represent a single process
class Process {
    int id;          // Process ID
    int arrivalTime; // When the process arrives
    int burstTime;   // Total CPU time needed
    int remainingTime; // CPU time still needed
    int priority;    // Priority level (lower number = higher priority)
    int waitTime;    // Time spent waiting
    int turnaroundTime; // Total time in system
    bool hasStarted; // Whether process has started executing
public:
    Process(int pid, int arrival, int burst, int prio) {
        id = pid;
        arrivalTime = arrival;
        burstTime = burst;
        remainingTime = burst;
```

```

    priority = prio;
    waitTime = 0;
    turnaroundTime = 0;
    hasStarted = false;
}
// Getters
int getId() const { return id; }
int getArrivalTime() const { return arrivalTime; }
int getBurstTime() const { return burstTime; }
int getRemainingTime() const { return remainingTime; }
int getPriority() const { return priority; }
int getWaitTime() const { return waitTime; }
int getTurnaroundTime() const { return turnaroundTime; }
bool getHasStarted() const { return hasStarted; }
// Process execution methods
void decrementRemainingTime() { remainingTime--; }
void markAsStarted() { hasStarted = true; }
void updateWaitTime(int time) {
    waitTime = time - arrivalTime - (burstTime - remainingTime); }
void setTurnaroundTime(int time) {
    turnaroundTime = time - arrivalTime;
}
bool isComplete() const { return remainingTime == 0; }
};
// Comparison functions for different scheduling algorithms
bool compareArrival(const Process& a, const Process& b) {
    return a.getArrivalTime() < b.getArrivalTime();
}
bool compareBurst(const Process& a, const Process& b) {
    return a.getBurstTime() < b.getBurstTime();
}
bool compareRemaining(const Process& a, const Process& b) {
    return a.getRemainingTime() < b.getRemainingTime();
}

```

```

}

bool comparePriority(const Process& a, const Process& b) {
    return a.getPriority() < b.getPriority(); }

// First Come First Serve (FCFS)

void firstComeFirstServe(vector<Process> processes) {
    cout << "\nRunning First Come First Serve Scheduler:\n";
    int currentTime = 0;
    int completed = 0;
    while (completed < processes.size()) {
        for (auto& process : processes) {
            if (process.getArrivalTime() <= currentTime && !process.isComplete()) {
                if (!process.getHasStarted()) {
                    cout << "Time " << currentTime << ": Starting Process " << process.getId() <<
endl;
                    process.markAsStarted();
                }
                process.decrementRemainingTime();
                currentTime++;
                if (process.isComplete()) {
                    process.updateWaitTime(currentTime);
                    process.setTurnaroundTime(currentTime);
                    cout << "Time " << currentTime << ": Completed Process " << process.getId()
<< " (Wait: " << process.getWaitTime()
<< "ms, Turnaround: " << process.getTurnaroundTime() << "ms)\n";
                    completed++; }
                break;
            }
        }
        if (completed < processes.size()) currentTime++;
    }
}

// Shortest Job First (SJF) - Non-preemptive

void shortestJobFirst(vector<Process> processes) {

```

```

cout << "\nRunning Shortest Job First Scheduler:\n";
int currentTime = 0;
int completed = 0;
while (completed < processes.size()) {
    vector<Process*> available;
// Find all available processes
    for (auto& process : processes) {
        if (process.getArrivalTime() <= currentTime && !process.isComplete()) {
            available.push_back(&process);
        }
    }
    if (!available.empty()) {
        // Sort by burst time
        sort(available.begin(), available.end(),
            [](Process* a, Process* b) { return a->getBurstTime() < b->getBurstTime(); });
        Process* current = available[0];
        if (!current->getHasStarted()) {
            cout << "Time " << currentTime << ": Starting Process " << current->getId() <<
endl;

            current->markAsStarted();
        }
        current->decrementRemainingTime();
        if (current->isComplete()) {
            current->updateWaitTime(currentTime + 1);
            current->setTurnaroundTime(currentTime + 1);
            cout << "Time " << currentTime + 1 << ": Completed Process " << current-
>getId()
                << " (Wait: " << current->getWaitTime()
                << "ms, Turnaround: " << current->getTurnaroundTime() << "ms)\n";
            completed++;
        }
    }
    currentTime++;
}

```

```

    }
}
// Shortest Remaining Time First (SRTF) - Preemptive
void shortestRemainingTime(vector<Process> processes) {
    cout << "\nRunning Shortest Remaining Time First Scheduler:\n";
    int currentTime = 0;
    int completed = 0;
    while (completed < processes.size()) {
        vector<Process*> available;
        // Find all available processes
        for (auto& process : processes) {
            if (process.getArrivalTime() <= currentTime && !process.isComplete()) {
                available.push_back(&process);
            }
        }
        if (!available.empty()) {
            // Sort by remaining time
            sort(available.begin(), available.end(),
                [](Process* a, Process* b) { return a->getRemainingTime() < b->getRemainingTime(); });
            Process* current = available[0];
            if (!current->getHasStarted()) {
                cout << "Time " << currentTime << ": Starting Process " << current->getId() << endl;
                current->markAsStarted();
            }
            current->decrementRemainingTime();
            if (current->isComplete()) {
                current->updateWaitTime(currentTime + 1);
                current->setTurnaroundTime(currentTime + 1);
                cout << "Time " << currentTime + 1 << ": Completed Process " << current->getId()
                    << " (Wait: " << current->getWaitTime()
                    << "ms, Turnaround: " << current->getTurnaroundTime() << "ms)\n";
            }
        }
        currentTime++;
        completed++;
    }
}

```



```

        completed++;
    }
}
currentTime++;
}
}
// Round Robin
void roundRobin(vector<Process> processes, int quantum) {
    cout << "\nRunning Round Robin Scheduler (Quantum=" << quantum << "):\n";
    int currentTime = 0;
    int completed = 0;
    queue<Process*> readyQueue;
    vector<bool> inQueue(processes.size(), false);
    while (completed < processes.size()) {
        // Add newly arrived processes to queue
        for (auto& process : processes) {
            int index = &process - &processes[0];
            if (process.getArrivalTime() <= currentTime && !inQueue[index] &&
!process.isComplete()) {
                readyQueue.push(&process);
                inQueue[index] = true;
            }
        }
    }
    if (!readyQueue.empty()) {
        Process* current = readyQueue.front();
        readyQueue.pop();
        inQueue[current - &processes[0]] = false;

        if (!current->getHasStarted()) {
            cout << "Time " << currentTime << ": Starting Process " << current->getId() <<
endl;
            current->markAsStarted();
        }
    }
}

```

```

// Execute for quantum or remaining time, whichever is smaller
int executeTime = min(quantum, current->getRemainingTime());
for (int i = 0; i < executeTime; i++) {
    current->decrementRemainingTime();
    currentTime++;
    for (auto& process : processes) {
        int index = &process - &processes[0];
        if (process.getArrivalTime() <= currentTime && !inQueue[index] &&
!process.isComplete()) {
            readyQueue.push(&process);
            inQueue[index] = true;
        }
    }
}
if (current->isComplete()) {
    current->updateWaitTime(currentTime);
    current->setTurnaroundTime(currentTime);
    cout << "Time " << currentTime << ": Completed Process " << current->getId()
        << " (Wait: " << current->getWaitTime()
        << "ms, Turnaround: " << current->getTurnaroundTime() << "ms)\n";
    completed++;
} else {
    readyQueue.push(current);
    inQueue[current - &processes[0]] = true;
}
} else {
    currentTime++;
}
} }

// Priority Scheduling (Preemptive)
void priorityScheduling(vector<Process> processes) {
    cout << "\nRunning Priority Scheduler:\n";
    int currentTime = 0;
    int completed = 0;

```

```

while (completed < processes.size()) {
    vector<Process*> available;
    // Find all available processes
    for (auto& process : processes) {
        if (process.getArrivalTime() <= currentTime && !process.isComplete()) {
            available.push_back(&process);
        }
    }
    if (!available.empty()) {

// Sort by priority (lower number = higher priority)
        sort(available.begin(), available.end(),
            [](Process* a, Process* b) { return a->getPriority() < b->getPriority(); });

        Process* current = available[0];
        if (!current->getHasStarted()) {
            cout << "Time " << currentTime << ": Starting Process " << current->getId() <<
endl;
            current->markAsStarted();
        }
        current->decrementRemainingTime();
        if (current->isComplete()) {
            current->updateWaitTime(currentTime + 1);
            current->setTurnaroundTime(currentTime + 1);
            cout << "Time " << currentTime + 1 << ": Completed Process " << current-
>getId()
                << " (Wait: " << current->getWaitTime()
                << "ms, Turnaround: " << current->getTurnaroundTime() << "ms)\n";
            completed++;
        }
    }
    currentTime++;
} }

```

```

int main() {
    // Create test processes: (ID, arrival time, burst time, priority)
    vector<Process> processes = {
        Process(1, 0, 8, 3),
        Process(2, 1, 4, 1),
        Process(3, 2, 9, 2),
        Process(4, 3, 5, 4),
        Process(5, 4, 2, 2)
    };
    // Test all scheduling algorithms
    vector<Process> p1 = processes;
    vector<Process> p2 = processes;
    vector<Process> p3 = processes;
    vector<Process> p4 = processes;
    vector<Process> p5 = processes;
    firstComeFirstServe(p1);
    shortestJobFirst(p2);
    shortestRemainingTime(p3);
    roundRobin(p4, 2); // Time quantum = 2
    priorityScheduling(p5);
    return 0;
}

```

5. Observations:

- **FCFS:** Simple but can lead to longer wait times due to the "convoy effect."
- **SJF & SRTF:** Minimize average wait and turnaround times but may cause starvation for longer processes.
- **Round Robin:** Fair and ensures responsiveness, though overhead increases with smaller quantum.
- **Priority Scheduling:** Efficient for priority-based tasks but can result in starvation of low-priority processes.

6. Recommendations:

- **FCFS** is suitable for batch processing or first-come-first-serve scenarios.
- **SJF** or **SRTF** is ideal for systems aiming for minimal response times.
- **Round Robin** works best in time-sharing systems where fairness and responsiveness are critical.
- **Priority Scheduling** is useful in real-time systems where specific tasks need precedence.

7. Results:

The simulation of the five scheduling algorithms yielded the following observations based on process metrics such as waiting time and turnaround time:

Algorithm	Average Waiting Time	Average Turnaround Time	Key Observations
FCFS	High	High	Simple but prone to the "convoy effect."
SJF	Low	Low	Efficient but may lead to starvation of longer tasks.
SRTF	Very Low	Very Low	Minimizes response times but increases context switches.
Round Robin	Moderate	Moderate	Fair and responsive but may have higher overhead.
Priority Scheduling	Varies	Varies	Effective for priority tasks but risks starvation of low-priority processes.

}

Running First Come First Serve Scheduler:

```
Time 0: Starting Process 1
Time 15: Completed Process 1 (Wait: 7ms, Turnaround: 15ms)
Time 16: Starting Process 2
Time 23: Completed Process 2 (Wait: 18ms, Turnaround: 22ms)
Time 24: Starting Process 3
Time 41: Completed Process 3 (Wait: 30ms, Turnaround: 39ms)
Time 42: Starting Process 4
Time 51: Completed Process 4 (Wait: 43ms, Turnaround: 48ms)
Time 52: Starting Process 5
Time 55: Completed Process 5 (Wait: 49ms, Turnaround: 51ms)
```

Running Shortest Job First Scheduler:

```
Time 0: Starting Process 1
Time 1: Starting Process 2
Time 4: Starting Process 5
Time 6: Completed Process 5 (Wait: 0ms, Turnaround: 2ms)
Time 7: Completed Process 2 (Wait: 2ms, Turnaround: 6ms)
Time 7: Starting Process 4
Time 12: Completed Process 4 (Wait: 4ms, Turnaround: 9ms)
Time 19: Completed Process 1 (Wait: 11ms, Turnaround: 19ms)
Time 19: Starting Process 3
Time 28: Completed Process 3 (Wait: 17ms, Turnaround: 26ms)
```

Running Shortest Remaining Time First Scheduler:

```
Time 0: Starting Process 1
Time 1: Starting Process 2
Time 5: Completed Process 2 (Wait: 0ms, Turnaround: 4ms)
Time 5: Starting Process 5
Time 7: Completed Process 5 (Wait: 1ms, Turnaround: 3ms)
Time 7: Starting Process 4
Time 12: Completed Process 4 (Wait: 4ms, Turnaround: 9ms)
Time 19: Completed Process 1 (Wait: 11ms, Turnaround: 19ms)
Time 19: Starting Process 3
Time 28: Completed Process 3 (Wait: 17ms, Turnaround: 26ms)
```

Running Round Robin Scheduler (Quantum=2):

```
Time 0: Starting Process 1
Time 4: Starting Process 2
Time 6: Starting Process 3
Time 12: Completed Process 1 (Wait: 4ms, Turnaround: 12ms)
Time 12: Starting Process 4
Time 14: Starting Process 5
Time 16: Completed Process 5 (Wait: 10ms, Turnaround: 12ms)
Time 16: Completed Process 1 (Wait: 8ms, Turnaround: 16ms)
Time 18: Completed Process 2 (Wait: 13ms, Turnaround: 17ms)
Time 18: Completed Process 2 (Wait: 13ms, Turnaround: 17ms)
```

Running Priority Scheduler:

```
Time 0: Starting Process 1
Time 1: Starting Process 2
Time 5: Completed Process 2 (Wait: 0ms, Turnaround: 4ms)
Time 5: Starting Process 3
Time 14: Completed Process 3 (Wait: 3ms, Turnaround: 12ms)
Time 14: Starting Process 5
Time 16: Completed Process 5 (Wait: 10ms, Turnaround: 12ms)
Time 23: Completed Process 1 (Wait: 15ms, Turnaround: 23ms)
Time 23: Starting Process 4
Time 28: Completed Process 4 (Wait: 20ms, Turnaround: 25ms)
```

8. Conclusion:

The study highlights the trade-offs between simplicity, efficiency, and fairness in CPU scheduling:

- **FCFS** is suitable for straightforward tasks but can result in long wait times.
- **SJF** and **SRTF** provide efficient resource utilization and low average wait times but may starve longer or lower-priority processes.
- **Round Robin** ensures fairness and responsiveness, making it ideal for time-sharing systems.
- **Priority Scheduling** caters to systems where task importance varies but requires mechanisms to prevent starvation.

Selecting the right scheduling algorithm depends on the system's workload and performance goals, balancing efficiency, fairness, and responsiveness.